UNIT-3

* Knowledge Engineering

Definition :- "Knowledge Engineering is the application of logic and ontology to the task of building computable models of some domain for some purpose".

- Engineering however, uses science and mathematics for the purpose of solving practical problems within the constraints of budgets and deadlines.
- Knowledge Engineering can therefore be defined as the branch of engineering that analyzes knowledge about some subjects and transforms it to a computable form for some purpose.

Example :-

A knowledge engineer models a "traffic light system" where the light alternates between red and green automatically (or) manually controlled under special conditions.

i, Translating informal and Formal Specifications:-

Informal is natural language descriptions must be translated into precise, computable formats.

(a) Informal Specification

(b) Formal Specification

(a) Informal Specifications :-

(It is a Questionary part it takes one task and analyse questions.)

Sentence written

The issues a knowledge engineer must address. Consider the problem of translating an informal English Specification to an executable program

Example Traffic System

There is a traffic light that automatically turns red (or) green, but it also has an option for mannual control under special circumstances.

- Imagine a Martian who had learned English by Monitoring radio broadcasts but who had never visited the earth.

" How would the martian interpret the terms automatically, option and mannual control ?"

- What does it mean for a light to "turn red (or) green ?
- Does it rotate ?
- How is the light related to the traffic ?

(b) Formalization Specifications :-

- A knowledge engineer who has mastered logic, frames and rule-based systems.
- A Sentence written by an expert in any of those fields is like Sentence about traffic light

- Every term that might seem "obvious" to the expert is a puzzle that the knowledge engineer must analyze and clarify.
- English syntax is not what makes the translation difficult. The difficulty results from the enormous amount of background knowledge.

Variables :- current color (Red/green), autoSwitch (on/off)

Rules :-

- If autoswitch = on, Green last 99 seconds, then turns red.
- If autoswitch = off, Manual inputs control the light.

ii) Principles of knowledge representation :-

Three experts in knowledge representation, "Randall Davis, Howard Schrobe, and peter" (1993) wrote a critical review and analysis of the state. their conclusions in five basic principles :-

(a) A knowledge representation is a surrogate
(b) A knowledge representation is a set of ontological commitments
(c) A knowledge representation is a fragmentary theory of intelligent reasoning
(d) A knowledge representation is a medium for efficient computation
(e)

the knowledge engineer must translate the detailed specification to a computable form.

PRINCIPLES OF KNOWLEDGE REPRESENTATION. Three experts in knowledge representation, Randall Davis, Howard Schrobe, and Peter Szolovits (1993), wrote a critical review and analysis of the state of the art. They summarized their conclusions in five basic principles about knowledge representations and their role in artificial intelligence:

1. *A knowledge representation is a surrogate.* Physical objects, events, and relationships, which cannot be stored directly in a computer, are represented by symbols that serve as *surrogates* for the external things. The symbols and the links between them form a model of the external system. By manipulating the internal surrogates, a computer program can simulate the external system or reason about it.
2. *A knowledge representation is a set of ontological commitments.* Ontology is the study of existence. For a database or knowledge base, ontology determines the categories of things that exist or may exist in an application domain. Those categories represent the *ontological commitments* of the designer or knowledge engineer.
3. *A knowledge representation is a fragmentary theory of intelligent reasoning.* To support reasoning about the things in a domain, a knowledge representation must also describe their behavior and interactions. The description constitutes a *theory* of the application domain. The theory may be stated in explicit axioms, or it might be compiled into executable programs. If the light is red now, it will turn green in rr seconds.
4. *A knowledge representation is a medium for efficient computation.* Besides representing knowledge, an AI system must encode knowledge in a form that can be processed efficiently on the available computing equipment. As Leibniz realized, some of the most interesting problems can be represented easily enough, but solving them may require an enormous amount of time and effort to compute. New developments in computer hardware and programming theory have had a major influence on the design and use of knowledge representation languages.
5. *A knowledge representation is a medium of human expression.* A good knowledge representation language should facilitate communication between the *knowledge engineers* who understand AI and the *domain experts* who understand the application. Although the knowledge engineers may write the definitions and rules, the domain experts should be able to read them and verify whether they represent a realistic theory of the domain.

Facilitates communication with domain experts.

These five principles can be used as a framework for discussing the issues of knowledge representation and illustrating them in terms of the traffic light example.

SURROGATE. A computational model is a surrogate for some real or hypothetical system — in this case, the traffic light. Each of the significant entities, such as the current time or the color of the light, is assigned a variable that serves as an internal surrogate for the external entity. Then the values of those variables can be transformed, either procedurally or declaratively, to simulate the behavior of the system. The procedural approach uses programs or rules that operate on the variables of the model $\mathcal{M}$:

1. Represent each possible event by an operation that changes $\mathcal{M}$ in an appropriate way.
2. Design a control structure that triggers each operation at the moment when its starting conditions become true.
3. Run the programs to see how the model $\mathcal{M}$ evolves in time.

Programs can be executed directly by a computer. Rules must be interpreted by some other program called an *inference engine*. For better performance, an optimizing compiler could be used to translate the rules to programs that run directly on the underlying computer.

The declarative approach is based on constraints or *axioms* that define the *preconditions*, *postconditions*, and transformations for each event that may occur in the model. Instead of executing programs to simulate the events, it uses theorem-proving techniques to derive their consequences. For any simulation that can be computed procedurally, the declarative approach can derive the same result by deduction:

1. Describe the starting state S by a collection of formulas.
2. Add the description of S to the axioms for the model $\mathcal{M}$.
3. To determine the state of $\mathcal{M}$ at any later time t, start with the axioms of $\mathcal{M}$ and the description of S, and prove a theorem that describes the state of $\mathcal{M}$ at time t.

In effect, the theorem prover serves as an interpreter that "executes" the axioms. The challenge, however, is to make it efficient. That is an engineering task of solving a problem within the constraints of budgets and deadlines.

ONTOLOGICAL COMMITMENTS. By Quine's criterion, the ontological commitments are determined by the types of variables in the knowledge representation.

For the traffic light example, the variables stand for the significant entities, such as the light itself, the current time, the current color, the last time when the color changed, the duration the light is red or green, and the switch for automatic control. Those commitments may be represented by variables in predicate calculus, by concepts in a graph, or by slots in data structures called *frames, schemas,* or *templates.* Following is a template for trafficLight, written in a rule-based language called CLIPS:

```
(deftemplate trafficLight
  (slot name              (type SYMBOL))
  (slot currentColor      (allowed-values red green))
  (slot redTime           (type FLOAT))
  (slot greenTime         (type FLOAT))
  (slot whenChanged       (type FLOAT))
  (slot autoSwitch        (allowed-values on off)) )
```

This template has six *slots,* each with a *facet* that restricts the values that may be stored in the slots. The template and its slots represent the primary ontological commitments. The facets add further commitments to the existence of entities of type Float and Symbol. Those entities do not belong to the object domain of traffic lights, but to the metalanguage domain used to talk about the symbols that represent traffic lights.

After the template has been defined, it can be *instantiated* by an *assert* statement, which inserts values in the slots. The autoSwitch slot would have the initial value *on,* but it could be turned off for manual control.

```
(assert (trafficLight
  (name              Blinky)
  (currentColor      green)
  (redTime           60)
  (greenTime         60)
  (whenChanged       0)
  (autoSwitch        on) ))
```

The template corresponds to a structure or record definition in a programming language. Each assertion for that template allocates and initializes the storage for a new instance. In logic, the slots correspond to dyadic predicates or relations such as redTime(x,y), where the first variable x represents the traffic light and the second variable y represents the current value in the slot.

MEDIUM FOR EFFICIENT COMPUTATION. Both the procedural and the declarative approaches can be transformed to a computable form. If the utmost in

efficiency is necessary for some problem, then a low-level programming language that maps directly to hardware may be the fastest. The following program simulates the traffic light by a loop that alternately sets the color to red or green:

```
loop while autoSwitch;
    set currentColor to red;
    wait redTime;
    set currentColor to green;
    wait greenTime;
    end loop;
```

This program could be used either to simulate a traffic light or to drive a microprocessor that controls an actual traffic light. As a knowledge representation, however, it lacks one essential feature: there is no way to get information out of it. The traffic light doesn't explain why it changes color, it doesn't keep a record of its color changes, and it doesn't answer questions about the average time it was red or green or was being operated under manual control.

If statistics about the traffic light are unimportant or irrelevant, then a programming loop may be the best way to simulate it. But if the traffic light itself is significant, then more instructions would be needed to keep records about the light and its changes. Those are *metalevel* instructions that support reasoning about the traffic light, but they could easily become much more voluminous and time-consuming than the original loop. To handle the detail, various *simulation languages* have been developed that operate on two levels in parallel: at the object level, they compute the effects of each action performed by the objects in the simulated system; at the metalevel, they monitor the objects and record information about them. The first modern *object-oriented language*, Simula-67, was originally designed for simulation. O-O languages are still popular for simulation, since the metalevel instructions can be *encapsulated* inside procedures associated with each object.

The declarative way to simulate the traffic light is to translate the English description to formulas in logic and to prove theorems about its states. Following are two axioms that represent the conditions for the light to turn green or red:

$$(\forall x{:}\text{TrafficLight})(\forall t{:}\text{Time})(\forall r{:}\text{Duration})$$
$$((\text{turnsRed}(x,t) \wedge \text{redTime}(x,r) \wedge \text{autoSwitch(on)})$$
$$\supset \text{turnsGreen}(x,t+r)).$$
$$(\forall x{:}\text{TrafficLight})(\forall t{:}\text{Time})(\forall g{:}\text{Duration})$$
$$((\text{turnsGreen}(x,t) \wedge \text{greenTime}(x,g) \wedge \text{autoSwitch(on)})$$
$$\supset \text{turnsRed}(x,t+g)).$$

The first axiom says that for any traffic light x, time t, and duration r, if x turns red at time t, the red time of x is r, and the autoswitch is turned on, then x turns

green at time $t + r$. The second axiom is the same, but with green and red interchanged.

Many different kinds of theorem provers have been developed for variations of logic and related declarative languages. Prolog was the first and is still the most popular *logic-programming language*. Following are the translations of the two axioms to Prolog rules:

```
turns_green(X,T2) ← traffic_light(X) & turns_red(X,T)
  & red_time(X,R) & auto_switch(on) & sum_times(T,R,T2).
turns_red(X,T2) ← traffic_light(X) & turns_green(X,T)
  & green_time(X,G) & auto_switch(on) & sum_times(T,G,T2).
```

These two rules are logically equivalent to the axioms, but with the syntactic changes necessary for Prolog: each variable begins with a capital letter; the quantifiers are omitted, since all Prolog variables are assumed to be universally quantified; and the implication operator $\supset$ is turned backward for the Prolog implication, which is represented by $\leftarrow$, <-, or :- in different dialects. The predicate sum_times would also have to be defined to make the proper conversions when seconds are added to hours and minutes.

Before these rules can be used computationally, the starting conditions for the traffic light would have to be asserted separately. The following four Prolog assertions correspond to the CLIPS assertion for the instance named Blinky (with the name written in lowercase to distinguish it from the Prolog variables).

```
traffic_light(blinky). red_time(blinky, 60).
green_time(blinky, 60). turns_green(blinky, 0:00).
```

These propositions assert that Blinky is a traffic light, it has a red time of 60 seconds, it has a green time of 60 seconds, and it turns green at time 0:00. From the two rules and the four assertions, a Prolog inference engine can deduce conclusions of the following form:

```
turns_red(blinky,0:01). turns_green(blinky,0:02).
turns_red(blinky,0:03). turns_green(blinky,0:04).
```

These conclusions would be generated in response to Prolog goals for the questions *When does Blinky turn red?* or *List all times when Blinky turns red or turns green.* Unlike the procedural loop, which continuously updates the variables to simulate the light, the Prolog rules make predictions about the light in answer to specific questions.

There are two major families of rule-based expert systems: the *backward-chaining* systems are supported by inference engines, which, like Prolog, are based on logical deduction; the *forward-chaining* systems are also based on logic, but they

have procedural aspects that resemble the programming loop. Following is a forward-chaining rule written in stylized English supplemented with variables:

```
if there is a trafficLight x,
  the currentColor of x is red,
  the redTime of x is r,
  the whenChanged of x is t,
  the autoSwitch is on,
  the currentTime is now,
  now ≥ t + r,
then modify currentColor of x to green,
  modify whenChanged of x to now.
```

This rule can be translated line by line to CLIPS, in which variables are marked by the ? symbol:

```
(defrule turnsGreen
  ?x <- (trafficLight
    (currentColor red)
    (redTime ?r)
    (whenChanged ?t)
    (autoSwitch on) )
  (currentTime ?now)
  (test (>= ?now (+ ?t ?r)))
=>
  (modify ?x
    (currentColor green)
    (whenChanged ?now) ))
```

The logic-based rules, which are true for all time ($\forall t$:Time), make predictions about when the light will turn red or green. The forward-chaining rule, however, is stated in terms of a constantly changing time called *now*. It lies dormant until the time $t+r$ becomes *now*; then, like the procedural loop, it modifies data in working memory. In effect, the if-part of the rule is as declarative as logic, and the then-part is as procedural as the loop. An inference engine must keep track of the changing patterns in working memory and activate each rule when all its conditions become true.

Fragmentary Theory of Reasoning. The procedural loop, the logical formula, and the forward-chaining rule illustrate three different strategies for reasoning about a traffic light: the instructions in the loop carry out a step-by-step simulation of the states of the light; the logical formulas support a theorem prover that can make a prediction about the light at any time t; and the forward-chaining rules, which lie

dormant until their conditions become true, spring into action to update the current state description. Any of these three reasoning strategies could also be used by humans who may be thinking about a traffic light. Each one has advantages and disadvantages, which may make it the preferred strategy for certain purposes.

The procedural approach is best suited to representing processes that have a natural sequence of operations. Each operation maps to a block of machine instructions, and the structure of the program mirrors the structure of the problem. But procedures are not as suitable for representing relationships where there is no natural time sequence. For such problems, the sequential nature of the procedure is an arbitrary imposition that may obscure or distort the structure of the problem. Even when time is significant, a pure procedural approach may still be unsuitable: a system with parallel operations and unforeseeable "surprises" can be difficult or impossible to map to a strictly sequential procedure.

Logic is best suited to describing patterns of relationships with criss-crossing dependencies that have no natural linearization. A graph logic is good for displaying such dependencies, but a linear notation such as predicate calculus can also express them in an equivalent form. To represent time sequences, logic must be supplemented with an appropriate ontology for time. Versions of *temporal logic* make time a privileged entity and explicitly represent the context-dependent time *now*. Parallel processes can also be represented in logic, but they require a context mechanism with message-passing operators for communicating between processes. Such mechanisms require further ontological assumptions about contexts, messages, and the axioms for relating them. They are described further in Chapters 4 and 5.

MEDIUM FOR HUMAN EXPRESSION. Since knowledge engineers must work with experts in other fields, they must be able to communicate with them in languages and notations that avoid the jargon of AI and computer science. Conceptual graphs and other diagrams have been used successfully as a communication medium between knowledge engineers and domain experts (Slagle et al. 1990). Figure 3.1 shows a conceptual graph equivalent to the logical formula or Prolog rule.

As Figure 3.1 illustrates, a conceptual graph may be more readable than linear rules, but the graphs are not self-explanatory. A domain expert who had never seen CGs before would need some explanation about the distinction between ovals and boxes, the directions of the arrows, and the conventions for coreference labels like *x and ?x. As an alternate notation, the conceptual graph could be translated to stylized English:

If a traffic light x turns red at time t,
  has a red time of a duration r,
  and has autoswitch in the state on,
then x turns green at a time, which is the sum of t and r.

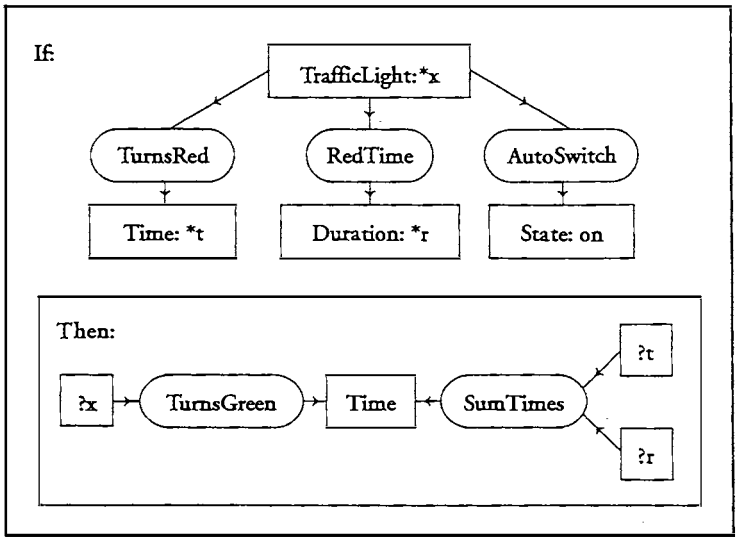

FIGURE 3.1 A conceptual graph for the traffic-light rule

This stylized English version requires less explanation, and it could be used as an alternate or a supplement to a formalized knowledge representation language. But specialized diagrams have become traditional in most fields: flow diagrams with tanks, pipes, and boilers in chemical engineering; circuit diagrams with transistors and resistors in electrical engineering; pieces on a board in chess; or notes and staves in music. Experts in any of those fields would prefer their familiar diagrams to any other notation, including natural language.

Translating an informal diagram to a formal notation of any kind is as difficult as translating informal English specifications to executable programs. But the converse can be much easier: a formal representation in conceptual graphs or any other system of logic can be translated automatically either to stylized natural language or to traditional kinds of diagrams. Walling Cyre and his students have developed a *scripting language* for translating conceptual graphs to circuit diagrams, block diagrams, and visual depictions (Cyre et al. 1994). With such tools, a logical specification written by the knowledge engineer could be translated automatically to visual diagrams for the domain expert, to executable code for the computer, or to natural language for documentation, explanations, and help facilities.

SIMULATION AND THEOREM PROVING. The procedural loop, the Prolog rules, and the forward-chaining rules represent three different ways of simulating the behavior of the traffic light. Yet simulation has certain limitations: it can only predict

what would happen in a specific case; it can never prove a general theorem. Logic has greater flexibility; it can be used to design a simulator, but it can also be used to prove theorems:

- Blinky turns green at every even-numbered minute.
- Blinky turns red at every odd-numbered minute.
- Blinky turns green 30 times per hour.
- Blinky will spend 50 percent of its time red.
- A traffic light with red time r and green time g will be red for $r/(r+g)$ of the time.
- A traffic light with red time r and green time g will turn green an average of $3600/(r+g)$ times per hour.

If the simulator were run for a long time, the results would suggest the patterns predicted by these theorems. Yet any patterns observed in the data might be caused by chance or by an unusual selection of starting conditions. Since the simulation was started at time 0, the theorem prover and the simulator would both show that Blinky turns green at every even-numbered minute. But only the theorem prover can explain how the results follow from the starting conditions.

Theorem proving can derive general principles with absolute certainty. But if a system is complex and changeable, the number of exceptions may make the rules so complex that finding a proof could be difficult or impossible. Even when a proof can be found, a simulation would be useful to check for possible errors in the starting assumptions or to give a survey of typical results. Ideally, simulation and theorem proving can supplement one another: patterns that are suggested by simulation can be verified by theorem proving.

The question of whether knowledge should be encoded declaratively in some form of logic or procedurally in a programming language is one of the oldest and still most hotly contested issues in AI and computer science in general. A declarative specification is usually the most concise and easiest to verify, but a low-level procedure is usually the most efficient. For programs that are only executed once, the human time is far more valuable than the computer time. If execution time is critical, optimizing compilers may be able to generate efficient code from declarative specifications. But for special purposes, suitable compilers might not be available, and some human programmer would have to translate the specifications to optimized code.

PERSISTENCE. Both the simulator and the theorem prover determine what happens at the discrete time points when the light changes color. But further information is needed to determine what happens in the intervals between color changes. The English statement that the light "remains green" or "remains red" is

not captured in the formal axioms. To determine the light's color in the intervening intervals, the axioms for the traffic light must be supplemented with two additional axioms of *persistence*:

$$(\forall x{:}\text{TrafficLight})(\forall t{:}\text{Time})(\forall t_1{:}\text{Time})(\forall r{:}\text{Duration})$$
$$((\text{turnsRed}(x,t_1) \wedge \text{redTime}(x,r)$$
$$\wedge\ (t=t_1 \vee (\text{before}(t_1,t) \wedge \text{before}(t,t_1+r)))\ )$$
$$\supset \text{color}(x,\text{red},t)\ ).$$

This axiom says that if the traffic light x turns red at time t_1 and its red time is r, then at any time t in the interval (t_1,t_1+r) its color is red. The next axiom states the equivalent conditions for green:

$$(\forall x{:}\text{TrafficLight})(\forall t{:}\text{Time})(\forall t_1{:}\text{Time})(\forall g{:}\text{Duration})$$
$$((\text{turnsGreen}(x,t_1) \wedge \text{greenTime}(x,g)$$
$$\wedge\ (t=t_1 \vee (\text{before}(t_1,t) \wedge \text{before}(t,t_1+g)))\ )$$
$$\supset \text{color}(x,\text{green},t)\ ).$$

Axioms of persistence are a cumbersome way of saying something that should be obvious. Unfortunately, computers cannot recognize the obvious unless they are explicitly told how. These cumbersome axioms are a special-case solution to the more general *frame problem*, which is discussed in Section 4.7. The general approach to that problem is based on Leibniz's principle of *sufficient reason*: Nothing at all can happen without some reason (*Nihil omnino fit sine aliqua ratione*). Leibniz's principle is a metalevel axiom that is used to generate one or more lower-level axioms for each particular example. In this case, it implies that the light's color should remain unchanged unless some reason for change can be found.

3.2 Representing Structure in Frames

Besides representing knowledge, a language must be able to analyze knowledge in low-level primitives and organize it in high-level structures. In natural language, the basic unit is the word, and the basic structure is the sentence. But higher-level structures like paragraphs, sections, chapters, and volumes are needed to classify and organize the knowledge. In symbolic logic, the basic units are predicates, which are connected by operators like $\wedge$, $\vee$, and $\supset$ to create formulas. In conceptual graphs, the basic units are concepts and relations, which are linked to one another in graphs. Besides linking, a knowledge representation language must have methods of grouping or nesting that can organize knowledge and package it in larger structures.

SCHEMATA. Aristotle used the term *schema* for his patterns of valid syllogisms. That word, with its plural *schemata* (accent on the first syllable), has been adopted by philosophers and psychologists for the patterns that organize elements